```
$text = 'brown bread, green apples, orange er... oranges';
preg_match_all('/brown|green|orange/', $text, $matches);
print_r($matches[0]);
```

```
Output:
Array
(
    [0] => brown
    [1] => green
    [2] => orange
    [3] => orange
)
```

By using three different patterns, separated by |, we're able to match in three different ways, and we can extract matches for the three colors in the pattern. "Orange" occurs twice in the matches array because why not name a fruit after a color? (Or is it the other way around?) Alternate patterns are applied to the sample in the order they're defined, left-to-right. They become more useful when combined with the following feature…

Sub-patterns

By adding parentheses - () to an expression, it is possible achieve two outcomes: localizing a set of alternatives and capturing matching parts of the sample. These parenthesized expression components are called *sub-patterns* or sometimes *groups*. Captured sub-patterns/groups are made available to the caller (i.e., through the $matches pass-by-reference array we've been using in recent examples); they can also be used later in the pattern via an escape sequence, which we'll be looking at very shortly.

```
preg_match_all('/(brown|green|orange) apples/', $text, $matches);
print_r($matches);
```

```
Output:
Array
(
    [0] => Array
```